

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.1: Computational Complexity and Memory Bounds

Theorem 1: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 1.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.2: Computational Complexity and Memory Bounds

Theorem 2: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 2.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.3: Computational Complexity and Memory Bounds

Theorem 3: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 3.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.4: Computational Complexity and Memory Bounds

Theorem 4: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 4.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.5: Computational Complexity and Memory Bounds

Theorem 5: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 5.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.6: Computational Complexity and Memory Bounds

Theorem 6: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 6.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.7: Computational Complexity and Memory Bounds

Theorem 7: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 7.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.8: Computational Complexity and Memory Bounds

Theorem 8: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 8.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.9: Computational Complexity and Memory Bounds

Theorem 9: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 9.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 10: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 10.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 11: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 11.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 12: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 12.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 13: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 13.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 14: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 14.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 15: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 15.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 16: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 16.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 17: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 17.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 18: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 18.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 19: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 19.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 20: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 20.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 1: Foundations of Discrete Algorithms and Computation

Section 1.21: Computational Complexity and Memory Bounds

Theorem 21: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 21.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 22: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 22.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 23: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 23.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 24: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 24.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 25: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 25.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 26: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 26.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 27: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 27.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 28: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 28.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 29: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 29.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 30: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 30.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 31: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 31.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 32: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 32.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 33: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 33.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 34: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 34.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 35: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 35.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 36: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 36.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 37: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 37.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 38: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 38.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 39: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 39.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 40: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 40.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 41: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 41.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 42: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 42.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 43: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 43.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 44: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 44.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 45: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 45.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 46: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 46.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 47: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 47.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 48: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 48.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 49: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 49.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 50: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 50.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 51: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 51.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 52: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 52.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 53: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 53.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 54: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 54.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 55: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 55.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 56: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 56.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 57: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 57.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 58: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 58.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 59: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 59.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 60: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 60.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 61: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 61.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 62: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 62.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 63: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 63.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 64: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 64.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 65: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 65.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 66: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 66.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 67: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 67.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 68: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 68.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 69: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 69.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 70: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 70.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 71: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 71.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 72: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 72.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 73: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 73.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 74: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 74.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 75: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 75.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 76: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 76.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 77: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 77.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 78: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 78.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 79: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 79.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 80: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 80.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 81: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 81.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 82: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 82.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 83: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 83.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 84: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 84.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 4: Foundations of Discrete Algorithms and Computation

Section 4.1: Computational Complexity and Memory Bounds

Theorem 85: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 85.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 86: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 86.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 87: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 87.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 88: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 88.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 89: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 89.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 90: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 90.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 91: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 91.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 92: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 92.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 93: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 93.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 94: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 94.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 95: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 95.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 96: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 96.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 97: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 97.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 98: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 98.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 99: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 99.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 100: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 100.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 101: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 101.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 102: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 102.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 103: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 103.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 104: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 104.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 105: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 105.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 106: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 106.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 107: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 107.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 108: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 108.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 109: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 109.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 110: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 110.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 111: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 111.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 112: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 112.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 113: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 113.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 114: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 114.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 115: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 115.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 116: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 116.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 117: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 117.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 118: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 118.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 119: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 119.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 120: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 120.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 121: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 121.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 122: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 122.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 123: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 123.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 124: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 124.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 125: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 125.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 126: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 126.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 127: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 127.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 128: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 128.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 129: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 129.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 130: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 130.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 131: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 131.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 132: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 132.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 133: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 133.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 134: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 134.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 135: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 135.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 136: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 136.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 137: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 137.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 138: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 138.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 139: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 139.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 140: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 140.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 141: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 141.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 142: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 142.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 143: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 143.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 144: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 144.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 145: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 145.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 146: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 146.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 147: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 147.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 148: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 148.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 149: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 149.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 150: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 150.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 151: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 151.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 152: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 152.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 153: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 153.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 154: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 154.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 155: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 155.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 156: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 156.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 157: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 157.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 158: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 158.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 159: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 159.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 160: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 160.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 161: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 161.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 162: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 162.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 163: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 163.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 164: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 164.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 165: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 165.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 166: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 166.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 167: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 167.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 168: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 168.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 169: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 169.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 170: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 170.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 171: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 171.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 172: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 172.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 173: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 173.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 174: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 174.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 175: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 175.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 176: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 176.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 177: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 177.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 178: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 178.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 179: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 179.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 180: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 180.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 181: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 181.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 182: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 182.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 183: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 183.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 184: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 184.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 185: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 185.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 186: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 186.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 187: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 187.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 188: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 188.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 189: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 189.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 190: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 190.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 191: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 191.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 192: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 192.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 193: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 193.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 194: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 194.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 195: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 195.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 196: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 196.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 8: Foundations of Discrete Algorithms and Computation

Section 8.1: Computational Complexity and Memory Bounds

Theorem 197: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 197.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 198: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 198.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 199: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 199.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 200: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 200.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 201: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 201.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 202: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 202.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 203: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 203.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 204: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 204.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 205: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 205.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 206: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 206.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 207: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 207.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 208: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 208.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 209: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 209.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 210: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 210.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 211: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 211.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 212: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 212.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 213: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 213.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 214: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 214.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 215: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 215.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 216: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 216.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 217: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 217.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 218: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 218.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 219: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 219.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 220: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 220.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 221: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 221.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 222: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 222.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 223: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 223.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 224: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 224.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 9: Foundations of Discrete Algorithms and Computation

Section 9.1: Computational Complexity and Memory Bounds

Theorem 225: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 225.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 226: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 226.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 227: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 227.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 228: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 228.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 229: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 229.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 230: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 230.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 231: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 231.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 232: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 232.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 233: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 233.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 234: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 234.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 235: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 235.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 236: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 236.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 237: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 237.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 238: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 238.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 239: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 239.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 240: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 240.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 241: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 241.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 242: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 242.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 243: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 243.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 244: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 244.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 245: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 245.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 246: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 246.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 247: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 247.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 248: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 248.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 249: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 249.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 250: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 250.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 251: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 251.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 252: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 252.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 10: Foundations of Discrete Algorithms and Computation

Section 10.1: Computational Complexity and Memory Bounds

Theorem 253: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 253.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 254: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 254.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 10: Foundations of Discrete Algorithms and Computation

Section 10.3: Computational Complexity and Memory Bounds

Theorem 255: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 255.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 256: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 256.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 257: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 257.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 258: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 258.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 259: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 259.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 260: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 260.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 261: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 261.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 262: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 262.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 263: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 263.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 264: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 264.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 265: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 265.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 266: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 266.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 267: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 267.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 268: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 268.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 269: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 269.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 270: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 270.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 271: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 271.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 272: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 272.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 273: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 273.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 274: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 274.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 275: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 275.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 276: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 276.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 277: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 277.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 278: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 278.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 279: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 279.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 280: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 280.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.1: Computational Complexity and Memory Bounds

Theorem 281: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 281.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.2: Computational Complexity and Memory Bounds

Theorem 282: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 282.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.3: Computational Complexity and Memory Bounds

Theorem 283: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 283.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.4: Computational Complexity and Memory Bounds

Theorem 284: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 284.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.5: Computational Complexity and Memory Bounds

Theorem 285: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 285.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.6: Computational Complexity and Memory Bounds

Theorem 286: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 286.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.7: Computational Complexity and Memory Bounds

Theorem 287: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 287.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.8: Computational Complexity and Memory Bounds

Theorem 288: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 288.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 289: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 289.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.10: Computational Complexity and Memory Bounds

Theorem 290: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 290.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.11: Computational Complexity and Memory Bounds

Theorem 291: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 291.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.12: Computational Complexity and Memory Bounds

Theorem 292: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 292.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.13: Computational Complexity and Memory Bounds

Theorem 293: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 293.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.14: Computational Complexity and Memory Bounds

Theorem 294: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 294.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.15: Computational Complexity and Memory Bounds

Theorem 295: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 295.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.16: Computational Complexity and Memory Bounds

Theorem 296: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 296.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.17: Computational Complexity and Memory Bounds

Theorem 297: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 297.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.18: Computational Complexity and Memory Bounds

Theorem 298: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 298.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.19: Computational Complexity and Memory Bounds

Theorem 299: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 299.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 300: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 300.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.21: Computational Complexity and Memory Bounds

Theorem 301: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 301.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.22: Computational Complexity and Memory Bounds

Theorem 302: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 302.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.23: Computational Complexity and Memory Bounds

Theorem 303: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 303.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 304: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 304.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.25: Computational Complexity and Memory Bounds

Theorem 305: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 305.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.26: Computational Complexity and Memory Bounds

Theorem 306: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 306.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 11: Foundations of Discrete Algorithms and Computation

Section 11.27: Computational Complexity and Memory Bounds

Theorem 307: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 307.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 308: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 308.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 12: Foundations of Discrete Algorithms and Computation

Section 12.1: Computational Complexity and Memory Bounds

Theorem 309: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 309.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 12: Foundations of Discrete Algorithms and Computation

Section 12.2: Computational Complexity and Memory Bounds

Theorem 310: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 310.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 12: Foundations of Discrete Algorithms and Computation

Section 12.3: Computational Complexity and Memory Bounds

Theorem 311: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 311.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 12: Foundations of Discrete Algorithms and Computation

Section 12.4: Computational Complexity and Memory Bounds

Theorem 312: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 312.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 313: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 313.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 314: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 314.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 315: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 315.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 316: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 316.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 317: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 317.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 318: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 318.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 319: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 319.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 320: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 320.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 321: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 321.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 322: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 322.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 12: Foundations of Discrete Algorithms and Computation

Section 12.15: Computational Complexity and Memory Bounds

Theorem 323: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 323.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 324: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 324.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 325: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 325.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 326: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 326.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 327: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 327.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 328: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 328.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 329: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 329.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 330: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 330.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 331: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 331.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 332: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 332.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 333: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 333.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 334: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 334.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 335: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 335.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 336: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 336.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 13: Foundations of Discrete Algorithms and Computation

Section 13.1: Computational Complexity and Memory Bounds

Theorem 337: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 337.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 13: Foundations of Discrete Algorithms and Computation

Section 13.2: Computational Complexity and Memory Bounds

Theorem 338: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 338.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 13: Foundations of Discrete Algorithms and Computation

Section 13.3: Computational Complexity and Memory Bounds

Theorem 339: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 339.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 340: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 340.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 341: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 341.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 342: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 342.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 13: Foundations of Discrete Algorithms and Computation

Section 13.7: Computational Complexity and Memory Bounds

Theorem 343: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 343.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 344: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 344.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 345: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 345.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 346: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 346.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 347: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 347.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 348: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 348.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 349: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 349.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 350: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 350.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 13: Foundations of Discrete Algorithms and Computation

Section 13.15: Computational Complexity and Memory Bounds

Theorem 351: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 351.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 352: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 352.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 353: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 353.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 354: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 354.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 355: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 355.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 356: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 356.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 357: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 357.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 358: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 358.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 359: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 359.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 360: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 360.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 361: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 361.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 362: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 362.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 363: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 363.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 364: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 364.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.1: Computational Complexity and Memory Bounds

Theorem 365: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 365.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.2: Computational Complexity and Memory Bounds

Theorem 366: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 366.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 367: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 367.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.4: Computational Complexity and Memory Bounds

Theorem 368: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 368.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 369: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 369.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.6: Computational Complexity and Memory Bounds

Theorem 370: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 370.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 371: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 371.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 372: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 372.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.9: Computational Complexity and Memory Bounds

Theorem 373: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 373.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 374: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 374.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.11: Computational Complexity and Memory Bounds

Theorem 375: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 375.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.12: Computational Complexity and Memory Bounds

Theorem 376: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 376.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 377: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 377.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 378: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 378.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 379: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 379.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 380: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 380.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 381: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 381.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 382: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 382.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 383: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 383.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 384: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 384.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 14: Foundations of Discrete Algorithms and Computation

Section 14.21: Computational Complexity and Memory Bounds

Theorem 385: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 385.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 386: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 386.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 387: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 387.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 388: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 388.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 389: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 389.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 390: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 390.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 391: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 391.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 392: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 392.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 15: Foundations of Discrete Algorithms and Computation

Section 15.1: Computational Complexity and Memory Bounds

Theorem 393: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 393.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 394: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 394.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 395: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 395.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 396: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 396.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 397: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 397.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 398: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 398.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 399: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 399.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 400: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 400.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 401: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 401.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 402: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 402.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 403: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 403.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 404: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 404.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 405: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 405.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 406: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 406.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 407: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 407.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 408: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 408.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 409: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 409.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 410: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 410.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 411: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 411.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 412: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 412.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 413: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 413.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 414: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 414.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 415: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 415.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 416: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 416.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 417: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 417.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 418: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 418.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 419: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 419.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 420: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 420.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 16: Foundations of Discrete Algorithms and Computation

Section 16.1: Computational Complexity and Memory Bounds

Theorem 421: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 421.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 16: Foundations of Discrete Algorithms and Computation

Section 16.2: Computational Complexity and Memory Bounds

Theorem 422: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 422.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 16: Foundations of Discrete Algorithms and Computation

Section 16.3: Computational Complexity and Memory Bounds

Theorem 423: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 423.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 424: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 424.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 425: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 425.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 426: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 426.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 427: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 427.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 428: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 428.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 429: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 429.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 430: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 430.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 431: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 431.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 432: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 432.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 433: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 433.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 434: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 434.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 435: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 435.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 436: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 436.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 437: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 437.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 438: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 438.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 439: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 439.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 440: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 440.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 441: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 441.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 442: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 442.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 443: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 443.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 444: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 444.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 445: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 445.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 446: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 446.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 447: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 447.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 448: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 448.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 17: Foundations of Discrete Algorithms and Computation

Section 17.1: Computational Complexity and Memory Bounds

Theorem 449: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 449.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 450: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 450.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 451: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 451.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 17: Foundations of Discrete Algorithms and Computation

Section 17.4: Computational Complexity and Memory Bounds

Theorem 452: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 452.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 453: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 453.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 454: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 454.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 455: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 455.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 456: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 456.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 457: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 457.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 458: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 458.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 459: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 459.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 460: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 460.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 461: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 461.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 462: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 462.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 463: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 463.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 464: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 464.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 465: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 465.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 466: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 466.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 467: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 467.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 468: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 468.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 469: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 469.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 470: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 470.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 471: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 471.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 472: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 472.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 473: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 473.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 474: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 474.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 475: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 475.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 476: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 476.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 18: Foundations of Discrete Algorithms and Computation

Section 18.1: Computational Complexity and Memory Bounds

Theorem 477: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 477.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 18: Foundations of Discrete Algorithms and Computation

Section 18.2: Computational Complexity and Memory Bounds

Theorem 478: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 478.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 479: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 479.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 480: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 480.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 481: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 481.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 482: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 482.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 18: Foundations of Discrete Algorithms and Computation

Section 18.7: Computational Complexity and Memory Bounds

Theorem 483: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 483.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 484: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 484.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 485: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 485.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 486: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 486.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 487: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 487.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 488: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 488.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 489: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 489.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 490: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 490.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 491: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 491.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 492: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 492.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 493: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 493.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 494: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 494.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 495: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 495.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 496: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 496.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 497: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 497.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 498: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 498.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 499: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 499.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 500: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 500.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 501: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 501.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 502: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 502.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 503: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 503.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 504: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 504.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 19: Foundations of Discrete Algorithms and Computation

Section 19.1: Computational Complexity and Memory Bounds

Theorem 505: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 505.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 506: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 506.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 507: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 507.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 508: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 508.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 509: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 509.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 19: Foundations of Discrete Algorithms and Computation

Section 19.6: Computational Complexity and Memory Bounds

Theorem 510: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 510.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 511: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 511.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 512: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 512.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 513: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 513.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 514: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 514.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 515: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 515.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 516: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 516.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 517: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 517.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 518: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 518.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 519: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 519.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 520: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 520.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 521: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 521.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 522: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 522.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 523: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 523.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 524: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 524.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 525: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 525.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 526: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 526.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 527: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 527.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 528: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 528.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 529: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 529.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 530: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 530.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 531: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 531.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 532: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 532.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 533: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 533.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 534: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 534.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 535: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 535.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 536: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 536.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 537: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 537.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 538: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 538.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 539: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 539.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 540: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 540.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 541: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 541.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 542: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 542.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 543: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 543.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 544: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 544.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 545: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 545.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 546: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 546.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 547: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 547.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 548: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 548.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 549: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 549.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 550: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 550.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 551: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 551.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 552: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 552.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 553: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 553.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 554: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 554.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 555: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 555.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 556: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 556.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 557: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 557.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 558: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 558.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 559: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 559.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 560: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 560.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 21: Foundations of Discrete Algorithms and Computation

Section 21.1: Computational Complexity and Memory Bounds

Theorem 561: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 561.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 562: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 562.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 563: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 563.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 564: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 564.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 565: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 565.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 566: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 566.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 567: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 567.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 568: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 568.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 569: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 569.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 570: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 570.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 571: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 571.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 572: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 572.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 573: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 573.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 574: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 574.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 575: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 575.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 576: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 576.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 577: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 577.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 578: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 578.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 579: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 579.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 580: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 580.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 581: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 581.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 582: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 582.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 583: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 583.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 584: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 584.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 585: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 585.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 586: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 586.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 587: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 587.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 588: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 588.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 589: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 589.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 590: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 590.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 591: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 591.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 592: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 592.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 593: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 593.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 594: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 594.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 595: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 595.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 596: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 596.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 597: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 597.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 598: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 598.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 599: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 599.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 600: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 600.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 601: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 601.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 602: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 602.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 603: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 603.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 604: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 604.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 605: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 605.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 606: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 606.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 607: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 607.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 608: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 608.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 609: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 609.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 610: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 610.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 611: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 611.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 612: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 612.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 613: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 613.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 614: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 614.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 615: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 615.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 616: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 616.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 617: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 617.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 618: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 618.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 619: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 619.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 620: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 620.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 621: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 621.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 622: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 622.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 623: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 623.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 624: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 624.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 625: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 625.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 626: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 626.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 627: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 627.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 628: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 628.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 629: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 629.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 630: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 630.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 631: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 631.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 632: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 632.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 633: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 633.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 634: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 634.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 635: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 635.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 636: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 636.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 637: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 637.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 638: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 638.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 639: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 639.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 640: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 640.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 641: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 641.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 642: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 642.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 643: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 643.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 644: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 644.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 24: Foundations of Discrete Algorithms and Computation

Section 24.1: Computational Complexity and Memory Bounds

Theorem 645: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 645.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 646: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 646.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 647: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 647.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 648: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 648.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 649: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 649.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 650: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 650.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 651: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 651.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 652: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 652.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 653: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 653.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 654: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 654.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 655: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 655.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 656: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 656.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 657: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 657.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 658: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 658.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 659: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 659.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 660: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 660.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 661: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 661.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 662: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 662.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 663: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 663.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 664: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 664.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 665: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 665.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 666: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 666.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 667: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 667.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 668: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 668.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 669: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 669.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 670: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 670.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 671: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 671.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 672: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 672.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Chapter 25: Foundations of Discrete Algorithms and Computation

Section 25.1: Computational Complexity and Memory Bounds

Theorem 673: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 673.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 674: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 674.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 675: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 675.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 676: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 676.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 677: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 677.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 678: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 678.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 679: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 679.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 680: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 680.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 681: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 681.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 682: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 682.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 683: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 683.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 684: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 684.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 685: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 685.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 686: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 686.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 687: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 687.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 688: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 688.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 689: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 689.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 690: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 690.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 691: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 691.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 692: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 692.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 693: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 693.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 694: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 694.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 695: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 695.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 696: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 696.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 697: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 697.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 698: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 698.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 699: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 699.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.

Theorem 700: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof follows from the binary decision tree height of $n!$ permutations, where $\log_2(n!) = n \log_2(n) - O(n)$.

Exercise 700.1: Verify the asymptotic tight bound for recurrence $T(n) = 2T(n/2) + \Theta(n)$.

Solution: By Case 2 of the Master Theorem, the recurrence evaluates to $\Theta(n \log n)$.